

Design Opportunities — Week 1

What I found, and how it should change the system

Deliverable 2 · Design Review (20 minutes) · Conversational Memory Intelligence System · Ketan Juikar · June 2026

Agenda: (1) the finding that changed my week · (2) the change I am proposing · (3) the experiment that tests it · (4) three more decisions, briefly · (5) what I want you to attack.

1. The finding that changed my week

I started the week planning to look at search and ranking, because that is where my worst failures are. I finished it realising I had the order wrong.

I cannot honestly judge *any* search or ranking idea — I cannot say adopt or reject with a straight face — because every failure I described in Deliverable 1 is currently just a *claim*. I have not measured a single one of them.

And while I was not looking, the field quietly solved that problem for me. In 2026, memory systems have three shared benchmarks: LoCoMo, LongMemEval, and BEAM. Each one is a fixed set of long conversations with questions attached, so different systems can be compared fairly. And their question categories line up almost exactly with my failures.

My failure (from Deliverable 1)	The matching benchmark questions
F3 (stale memory) — the old fact beats the new one	Questions about time, and about facts that change
F4 (exact-fact miss) — it cannot find what it has	Simple lookups, and questions needing several steps
Cold start — nothing relevant exists	Questions where the right answer is "I don't know"
F0 (stateless) and F1 (history replay) — the whole reason memory exists	Questions spanning many sessions

So the most valuable thing I can do this week is not to pick a clever search trick. It is to adopt the **way of measuring** that turns all of Deliverable 1 from opinion into something I can actually prove or disprove.

2. The change I am proposing

The change: Adopt **LongMemEval** as this project's main test set, and **LoCoMo** as a second opinion. Wire it up *before* the baseline experiment in Deliverable 3, so that the baseline reports real numbers per category, instead of stories. Fix which AI model does the judging and which prompt it uses, and report both as part of the result.

This is big enough to deserve a proper decision record, because it sets the standard that every later component gets measured against. It does not change the architecture at all. It makes the architecture *testable*.

It also takes the risk out of the three decisions below, because each one now has a specific score it has to beat in order to earn its place.

3. The experiment

This is the first experiment I will run. It proves the change is worth making, and settles my biggest search question at the same time.

Element	What it means
What I think will happen	Searching three ways at once — by meaning, by exact keyword, and by entity — will beat meaning-only search on exact-fact questions and on questions needing several steps, without making lookups too slow.
The setup	Same data, same store. Option A: search by meaning only. Option B: search all three ways, put the scores on a common scale, and merge them.
What I measure	Accuracy on each question category. How long lookups take. How many tokens each question uses.
Success looks like	Option B wins on accuracy for exact facts and multi-step questions, and lookups still finish in under about 100 milliseconds.
Failure looks like	If Option B does not beat Option A even after tuning the weights, then combining searches is not worth its complexity, and I go back to meaning-based search plus a recency boost.

4. Three more decisions, briefly

ADOPT **Search several ways at once** (Mem0, April 2026). This confirms the hybrid design I had already sketched. I adopt the pattern, and the experiment above tunes it.

PROTOTYPE **Re-score the results with a second model** (Hindsight). Probably a real accuracy win. But it sits on the live request path, where the user is waiting, and it costs speed. I will not adopt it on faith. Measured accuracy against measured delay decides it.

DEFER **Store two dates on every fact** — when it was true, and when it was recorded (Zep's approach). This is the best answer to F3 (stale memory). Zep scores 63.8% against Mem0's 49.0% on the same benchmark, a real 15-point gap on handling facts that change.

But it is heavy: a whole graph database, plus the delay that graph processing adds to every question. So I am postponing it behind a named trigger. If my cheaper rule — weight recency, and let the newer, more confident memory win — fails the questions about facts that change, then the graph is the planned response. I refuse to build it before the simple option has been shown to fail.

5. What I want you to attack

I would far rather be wrong here than in Deliverable 6. So please push hard on these four.

- **Does the evidence apply to me at all?** Those benchmark numbers were measured on GPT-4o, with very large conversations. My first version is smaller and may run local models. Does that evidence transfer? And if I use a smaller slice of the benchmark, does that quietly break it?
- **Am I optimising for the wrong thing?** Is the benchmark's traffic actually anything like my real CommentHook and migration usage? Is a small hand-built set from my own transcripts a real guard against that — or am I just fooling myself?
- **Is postponing the graph discipline, or cowardice?** Am I being sensibly frugal, or am I avoiding the hard component because it is hard? What result would prove I postponed it wrongly?
- **Is the simpler thing enough?** For preferences specifically — which is my actual use case — would meaning-based search plus a recency boost match the three-way search, making all that extra complexity pointless?

This week in one line: Adopt the benchmarks and the three-way search. Prototype the re-scoring model and single-pass extraction. Postpone the two-date graph, behind a named trigger. Every one of these is tied to a real failure — and every "adopt" now has a score it must beat to keep its place.